
Arm CCA — Enabling Realm VM Boot and Early Console

Cloud Hypervisor Porting

Q1 Realm KVM Interface

(a) How did you find the right place to patch? Include any source (e.g., linux kernel code, RFC patch, ...) you used to identify the issue.

```
enable_cap -> ioctl(KVM_ENABLE_CAP())
```

```
# ./cloud-hypervisor \
> --kernel /cca/Image --disk path=/cca/guest-disk.img \
> --cpus boot=1 --memory size=512M \
> --cmdline "console=hvc0 root=/dev/vda2" \
> --platform arm_rme=on,measurement_algo=sha256 \
> --serial off --console tty
cloud-hypervisor: 56.380780ms: <vmm> WARN:vmm/src/memory_manager.rs:1192 -- We don't support guest memfd with
huge pages at the moment
cloud-hypervisor: 485.499610ms: <vmm> WARN:arch/src/aarch64/fdt.rs:113 -- File: /sys/devices/system/cpu/cpu0/c
ache/index3/size does not exist.
cloud-hypervisor: 486.214160ms: <vmm> WARN:arch/src/aarch64/fdt.rs:147 -- File: /sys/devices/system/cpu/cpu0/c
ache/index3/coherency_line_size does not exist.
cloud-hypervisor: 486.930320ms: <vmm> WARN:arch/src/aarch64/fdt.rs:170 -- File: /sys/devices/system/cpu/cpu0/c
ache/index3/number_of_sets does not exist.
cloud-hypervisor: 488.045860ms: <vmm> WARN:arch/src/aarch64/fdt.rs:428 -- L2 cache shared with other cpus
Error booting VM: VmBoot(CreateArmRme(ConfigRealm(Invalid argument (os error 22))))
```

I started from the error message to locate where in the source things could go wrong, which first points to `arm_rme_realm_create` in `cloud-hypervisor/hypervisor/src/kvm/mod.rs`. From the error message I could also tell which branch failed: it must be the `kvm_enable_cap` one, because we only get `os error 22` (i.e. `EINVAL`) with no extra message printed.

Next, from the spec + source code, I learned how the Rust wrappers are layered: `kvm-ioctls` defines the behavior (which `ioctl` to issue to KVM), while `kvm-bindings` defines the content of that `ioctl` and maps it onto `kvm.h` in Linux.

```

// Configure the realm and create the realm descriptor
fn arm_rme_realm_create(&self, realm_config: &ArmRmeConfig) -> vm::Result<()> {
    if let Some(rpv) = realm_config.personalization_value {
        let rpv_bytes = base64
            .decode(rpv)
            .map_err(|e| vm::HypervisorVmError::ConfigRealm(e.into()))?;

        if rpv_bytes.len() != ARM_RME_CONFIG_RPV_SIZE as usize {
            return Err(vm::HypervisorVmError::ConfigRealm( anyhow!(
                "invalid RPV length"
            )));
        }

        let mut cfg = arm_rme_config {
            cfg: ARM_RME_CONFIG_RPV,
            ..Default::default()
        };

        // Fill the first few bytes. The RPV is zero-padded on the right
        for (i, b) in rpv_bytes.into_iter().enumerate() {
            // SAFETY: accessing a union field in a valid structure
            unsafe {
                cfg.__bindgen_anon_1.__bindgen_anon_1.rpv[i] = b;
            }
        }

        let cap = kvm_enable_cap {
            cap: KVM_CAP_ARM_RME,
            args: [
                KVM_CAP_ARM_RME_CONFIG_REALM as u64,
                &cfg as *const _ as u64,
                0,
                0,
            ],
            ..Default::default()
        };

        self.fd
            .enable_cap(&cap) [7/19]
            .map_err(|e| vm::HypervisorVmError::ConfigRealm(e.into()))?
    }
}

```

Putting this together, I first checked whether the definition of cap: KVM_CAP_ARM_RME in the bindings matches the one in kvm.h:

- kvm.h

```

#define KVM_CAP_ARM_RME 240 /KVM_CAP_ARM_RME [1/2]

```

- bindings.rs

```

pub const KVM_CAP_ARM_RME: u32 = 300; // FIX v7 -> v8

```

kvm.h says 240, but bindings.rs says 300 -> root cause found. After changing both to 240, a Normal VM boots fine.

```

# ./cloud-hypervisor \
> --kernel /cca/Image \
> --disk path=/cca/guest-disk.img \
> --cpus boot=1 \
> --memory size=512M \
> --cmdline "console=hvc0 root=/dev/vda2" \
> --serial off \
> --console tty
cloud-hypervisor: 488.979580ms: <vmm> WARN:arch/src/aarch64/fdt.rs:113 -- File: /sys/devices/system/cpu/cpu0/cache/index3/size does not exist.
cloud-hypervisor: 489.694770ms: <vmm> WARN:arch/src/aarch64/fdt.rs:147 -- File: /sys/devices/system/cpu/cpu0/cache/index3/coherency_line_size does not exist.
cloud-hypervisor: 490.412860ms: <vmm> WARN:arch/src/aarch64/fdt.rs:170 -- File: /sys/devices/system/cpu/cpu0/cache/index3/number_of_sets does not exist.
cloud-hypervisor: 491.572170ms: <vmm> WARN:arch/src/aarch64/fdt.rs:428 -- L2 cache shared with other cpus
[ 1.443307] loop: module loaded
[ 1.444952] virtio_blk virtio1: 1/0/0 default/read/poll queues
[ 1.458959] virtio_blk virtio1: [vda] 258048 512-byte logical blocks (132 MB/126 MiB)
[ 1.473260] vda: vda1 vda2

```

```

Starting syslogd: OK
Starting klogd: OK
Running sysctl: OK
Starting tee-supplciant: Using device /dev/teepriv0.
ERR [84] TSUP:main:908: make_daemon(): -1
FAIL
Starting network: OK
Starting crond: OK

Welcome to Buildroot
buildroot login: root
# uname -a
Linux buildroot 6.15.0-rc1-g916aeec68dd4 #1 SMP PREEMPT @1779531200 aarch64 GNU/Linux

```

But with only the change above, the Realm VM still does not start.

```
Error booting VM: VmBoot(CpuManager(RecFinalize(VcpuFinalize(Invalid argument (os error 22))))))
```

The problem this time is in KVM_ARM_VCPU_FINALIZE. Following the same reasoning as the Normal VM, I could locate the root cause and change the constant, after which the Realm VM starts correctly.

- bindings.rs

```

6 | KVM_ARM_VCPU_REC: u32 = 8; // FIX v7 -> v8

```

- kvm.h

```

#define KVM_ARM_VCPU_REC 9 /* VCPU REC state as part of Realm */ /VCPU_REC

```

Both bugs are really the same class of problem: cloud-hypervisor pins the kvm fork at cca/v7, but the linux in cca-3world is already cca-host/v8, and the kernel UAPI was renumbered in between (v8 inserted new enums before KVM_CAP_ARM_RME / before KVM_ARM_VCPU_REC), so the constants in the bindings no longer line up with the kernel's kvm.h. The way to identify both was therefore to diff the constants in bindings.rs against kvm.h under ~/.shrinkwrap/build/source/cca-3world/linux:

- KVM_CAP_ARM_RME: bindings 300 → kvm.h (uapi/linux/kvm.h:944) 240
- KVM_ARM_VCPU_REC: bindings 8 → kvm.h 9 (v8 inserted KVM_ARM_VCPU_HAS_EL2_E2H0 = 8)

(b) What is the RMI used in CCA to execute a vCPU?

In the file vmm/src/cpu.rs, the function start_vcpu creates a thread, sets up the attributes of the vcpu, and does the vcpu run with KVM.

We can keep tracking the code to `linux/arch/arm64/kvm/arm.c`, the function `kvm_arch_vcpu_ioctl_run`

- part of the code. It would jump to `kvm_rec_enter`

```
if (vcpu_is_rec(vcpu))
    ret = kvm_rec_enter(vcpu);
else
    ret = kvm_arm_vcpu_enter_exit(vcpu);
```

BTW the `vcpu` in RMM is called the Realm Execution Context (REC).

We can see `kvm_rec_enter` in `linux/arch/arm64/kvm/rme.c` (around line 1477), and it calls `rmi_rec_enter`, which is the SMC wrapper defined in `linux/arch/arm64/include/asm/rmi_cmds.h`:

- `kvm_rec_enter`

```
int kvm_rec_enter(struct kvm_vcpu *vcpu)
{
    struct realm_rec *rec = &vcpu->arch.rec;

    switch (rec->run->exit.exit_reason) {
    case RMI_EXIT_HOST_CALL:
    case RMI_EXIT_PSCI:
        for (int i = 0; i < REC_RUN_GPRS; i++)
            rec->run->enter.gprs[i] = vcpu_get_reg(vcpu, i);
        break;
    case RMI_EXIT_RIPAS_CHANGE:
        kvm_complete_ripas_change(vcpu);
        break;
    }

    if (kvm_realm_state(vcpu->kvm) != REALM_STATE_ACTIVE)
        return -EINVAL;

    return rmi_rec_enter(virt_to_phys(rec->rec_page),
                        virt_to_phys(rec->run));
}
```

- `rmi_rec_enter`

```
rmi_rec_enter,
static inline int rmi_rec_enter(unsigned long rec, unsigned long run_ptr)
{
    struct arm_smccc_res res;

    arm_smccc_1_1_invoke(SMC_RMI_REC_ENTER, rec, run_ptr, &res);

    return res.a0;
}
```

So the RMI is `RMI_REC_ENTER`, issued by `rmi_rec_enter` via the `SMC_RMI_REC_ENTER` smc call (FID `0x015c`, from `linux/arch/arm64/include/asm/rmi_smc.h`). After the RMM runs this REC, it does an ERET back into the Realm's EL1 to run the guest code.

(c) What RMI(s) are used to create stage-2 mapping of the Realm guest memory?

In the file `linux/arch/arm64/kvm/rme.c`, the stage-2 mapping of protected guest memory is done by the following RMIs:

- `rmi_data_create` (SMC_RMI_DATA_CREATE, FID 0x0153): in `realm_create_protected_data_granule`, it populates a protected leaf entry with known content (copied from the host). This is what the realm uses to load the kernel image at boot.
- `rmi_data_create_unknown` (SMC_RMI_DATA_CREATE_UNKNOWN, FID 0x0154): in `realm_map_protected`, it creates a protected leaf entry for a page that the guest only faults in at runtime (its content is not visible to the host).
- `rmi_rtt_create` (SMC_RMI_RTT_CREATE, FID 0x015d): `realm_create_rtt_levels` calls it to build the intermediate RTT levels (Realm Translation Table, i.e. the non-leaf structure of the stage-2 page table). If either `rmi_data_create*` above returns `RMI_ERROR_RTT`, it means the RTT at the corresponding level is missing, so it first calls this to create it and then retries.

In short: `rmi_rtt_create` builds the “structure” (intermediate levels) of the page table, and `rmi_data_create` / `rmi_data_create_unknown` fill in the “leaves”. (The corresponding unprotected/shared half is mapped with `rmi_rtt_map_unprotected`, FID 0x015f.)

Q2. Error analysis

The following RMM log appears repeatedly:

```
[ rmm ] Inject Sync EA into current REC. FAR_EL2: 0xffffffff5fd018, ELR_EL2: 0xffffc23d55f15a54
```

(a) What are `FAR_EL2` and `ELR_EL2`?

- `ELR_EL2`, Exception Link Register

When an interrupt, hypervisor call (HVC), or abort targets EL2, the processor automatically saves the address of the instruction that caused (or was about to execute) the exception into `ELR_EL2`. When the hypervisor finishes handling the exception, it executes an ERET instruction, which jumps back to the address stored in `ELR_EL2` to resume normal execution.

- `FAR_EL2`, Fault Address Register

If an exception occurs because the CPU tried to access an invalid or restricted memory location, `FAR_EL2` logs the exact faulting virtual address. Operating systems or hypervisors read `FAR_EL2` to diagnose what went wrong and to take corrective action (like mapping memory pages or terminating a misbehaving virtual machine)

(b) The lowest 12 bits of `FAR_EL2` are 0x018. Which PL011 register does this offset correspond to? From `include/linux/amba/serial.h`

```
#define UART01x_FR < 0x18 > /* Flag register (Read only). */ W [1/1]
```

i.e. `#define UART01x_FR 0x18`, which is the UART Flag register (UARTFR). Before sending a character, `earlycon` polls the TXFF/BUSY bits of this register, which is why the lowest 12 bits of `FAR_EL2` in the SEA are 0x018.

(c) Look at the RMM source code at `./shrinkwrap/build/source/cca-3world/rmm`, what is the exact function in RMM that injects this error?

From `/cca-3world/rmm/runtime/core/inject_exp.c`

```

void inject_sync_idabort(unsigned long fsc)
{
    unsigned long esr_el2 = read_esr_el2();
    unsigned long far_el2 = read_far_el2();
    unsigned long elr_el2 = read_elr_el2();
    unsigned long spsr_el2 = read_spsr_el2();
    unsigned long vbar_el1 = read_vbar_el1();
    unsigned long sctlr_el1 = read_sctlr_el1();

    unsigned long esr_el1 = calc_esr_idabort(esr_el2, spsr_el2, fsc);
    bool sctlr2_ease_bit = ((read_sctlr2_el1_if_present() &
        SCTL2_ELx_EASE_BIT) != 0UL);
    unsigned long pc = calc_vector_entry(vbar_el1, spsr_el2,
        sctlr2_ease_bit);
    unsigned long spsr = calc_spsr(spsr_el2, sctlr_el1);

    write_far_el1(far_el2);
    write_elr_el1(elr_el2);
    write_spsr_el1(spsr_el2);
    write_esr_el1(esr_el1);
    write_elr_el1(pc);
    write_spsr_el1(spsr);

    INFO("Inject Sync EA into current REC. FAR_EL2: 0x%lx, ELR_EL2: 0x%lx\n",
        far_el2, elr_el2);
}

```

The log line Inject Sync EA into current REC. FAR_EL2: ... is printed by the INFO() at the end of this function, so the injecting function is inject_sync_idabort. Its caller is handle_data_abort in runtime/core/exit.c: when a REC exits to the RMM on a data abort and the IPA is determined to be protected with RIPAS = EMPTY, it calls inject_sync_idabort(ESR_EL2_ABORT_FSC_SEA) (exit.c:233). This matches the spec description in Q3(b).

Q3. RMM specification

Refer to the RMM specification:

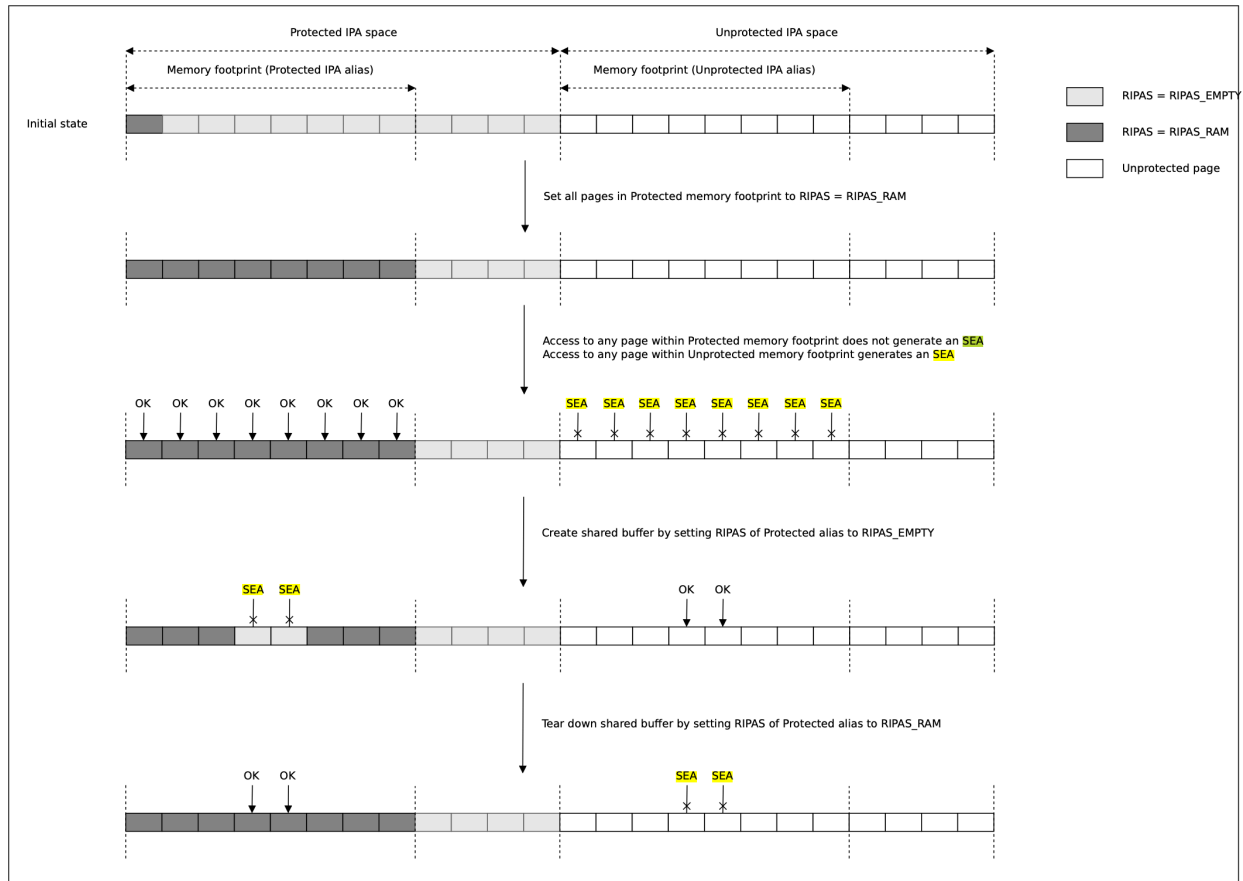
(a) Cite the spec to explain the distinction between unprotected IPA and protected IPA.

- From D2.1 Realm shared memory protocol description:

Realm software treats the most significant IPA bit as a “protection attribute” bit. This means that for every Protected IPA (in which the most significant bit is ‘0’), there exists a corresponding Unprotected IPA alias, which is generated by setting the most significant bit to ‘1’.

(b) Cite the spec to describe when the RMM injects an SEA into the REC.

From D2.1 Realm shared memory protocol flow:



From this flow we can summarize when the RMM injects an SEA into the REC: when the Realm accesses a Protected IPA whose RIPAS is EMPTY (i.e. it has not been set to RAM, so there is no actual backing protected page), the RMM injects the SEA back into that REC; conversely, accessing a protected page whose RIPAS is RAM does not generate an SEA. The third stage in the figure (access to the Unprotected footprint → SEA) and, when building a shared buffer, accessing the protected alias after setting it to RIPAS_EMPTY (→ SEA) are both expressions of the same rule.

This is consistent with the RMM source: in `runtime/core/exit.c::handle_data_abort`, when `empty_ipa = ipa_is_empty(fipa, rec)` is true it calls `inject_sync_idabort(ESR_EL2_ABORT_FSC_SEA)` (an IPA beyond the realm's IPA size also injects an SEA). The earlycon failure in this assignment is precisely because it hits the PL011 at the protected alias (top IPA bit = 0), and that protected IPA is EMPTY, which is why we see the repeated Inject Sync EA.

Q4. Verifying the fix

After applying your fix, run the following experiments on the same kernel image:

(a) `/proc/iomem` is exposed by the kernel, showing the current map of the system's physical memory and device MMIO regions. In the realm guest, run `grep "pl011" /proc/iomem`. What is the physical address of pl011?

```
# grep "pl011" /proc/iomem
09000000-09000fff : pl011@9000000
09000000-09000fff : 9000000.pl011 pl011@9000000
```

The physical address of pl011 is 0x09000000 (range 09000000-09000fff). This is the bare IPA given by the reg in the device tree (no shared bit set), so /proc/iomem shows the address in the protected half.

(b) Find the physical MMIO address of earlycon in dmesg. What is the address? Is it the same as the address you see in (a)?

```
[ 0.000000] earlycon: pl11 at MMIO 0x0000800009000000 (options '')
[ 0.000000] printk: legacy bootconsole [pl11] enabled
```

The earlycon MMIO address is 0x0000800009000000 (earlycon: pl11 at MMIO 0x0000800009000000).

No, the address is not the same as in (a). The difference is exactly bit 47 (the shared/unprotected bit): $0x800009000000 = 0x09000000 \mid (1 \ll 47)$. In Part 2 we OR the shared bit into the earlycon= address in cloud-hypervisor for the arm_rme case, so that earlycon points directly at the unprotected alias and can therefore print its messages successfully.

We did not change the device's reg in the device tree, so the device still reports its IPA in the same way (bare 0x09000000, i.e. what we see in (a)). The PL011 driver also works because the realm-aware ioremap() adds the shared bit in the page table for it.

In short, the difference is that the shared bit is applied at a different stage. earlycon -> we pre-add the shared bit to the address on the command line (before / bypassing the realm ioremap); the normal PL011 driver -> it is added by ioremap when the mapping is built (after ioremap).

(c) Identify two boot messages that appear exclusively when earlycon is enabled during the booting process, excluding the earlycon initialization itself.

```
# dmesg | grep -i bootconsole
[ 0.000000] printk: legacy bootconsole [pl11] enabled
[ 1.520865] printk: legacy bootconsole [pl11] disabled
```

Excluding earlycon's own initialization (earlycon: pl11 at MMIO 0x...), the two lines that appear only when earlycon is enabled are:

- printk: legacy bootconsole [pl11] enabled
- printk: legacy bootconsole [pl11] disabled

The former is when the bootconsole is registered, and the latter is when the real console (hvc0/ttyAMA0) takes over and hands off / disables the bootconsole. Without earlycon there is no bootconsole at all, so these two lines never appear.

(d) Does your patched cloud-hypervisor still produce a working earlycon for the Normal VM?

Yes, you can see the log in the bottom of picture.

```
# cd /cca
# ./cloud-hypervisor \
> --kernel /cca/Image \
> --disk path=/cca/guest-disk.img \
> --cpus boot=1 \
> --memory size=512M \
> --cmdline "console=ttyAMA0 root=/dev/vda2" \
> --serial tty \
> --console off
cloud-hypervisor: 483.987930ms: <vmm> WARN:arch/src/aarch64/fdt.rs:113 -- File: /sys/devices/system/cpu/cpu0/cache/index3/size does not exist.
cloud-hypervisor: 484.740800ms: <vmm> WARN:arch/src/aarch64/fdt.rs:147 -- File: /sys/devices/system/cpu/cpu0/cache/index3/coherency_line_size does not exist.
cloud-hypervisor: 485.496280ms: <vmm> WARN:arch/src/aarch64/fdt.rs:170 -- File: /sys/devices/system/cpu/cpu0/cache/index3/number_of_sets does not exist.
cloud-hypervisor: 486.637540ms: <vmm> WARN:arch/src/aarch64/fdt.rs:428 -- L2 cache shared with other cpus
[ 0.000000] Booting Linux on physical CPU 0x000000000 [0x410fd0f0]
[ 0.000000] Linux version 6.15.0-rc1-g916aee6c68dd4 (tuxmake@shrinkwrap) (aarch64-linux-gnu-gcc (Debian 14.2.0-19) 14.2.0, GNU ld (GNU Binutils for Debian) 2.44) #1 SMP PREEMPT @1779531200
[ 0.000000] KASLR enabled
[ 0.000000] Machine model: linux,dummy-virt
[ 0.000000] earlycon: pl11 at MMIO 0x0000000009000000 (options '')
[ 0.000000] printk: legacy bootconsole [pl11] enabled
```

Q5. Realm guest kernel: ioremap path

(a) Why do we `ioremap()` to use MMIO? Since we are already in the kernel, why can't we access the physical address of the device directly?

Once the MMU is enabled, the CPU issues only virtual addresses: every load/store is translated from a virtual address to a physical one, so the kernel can no longer dereference a physical address directly. Furthermore, the kernel's linear (direct) mapping only covers normal RAM — a device's MMIO region is deliberately left out of it, so no kernel virtual address points at the device to begin with. MMIO must also be mapped with device memory attributes (non-cacheable, no speculation/reordering/write-merging) rather than normal cacheable memory, or accesses would be cached or reordered and break the device's register semantics. `ioremap()` solves both problems: it allocates a kernel virtual address and installs a page-table mapping to the device's physical region with the correct device attributes, giving the kernel a usable pointer to the MMIO.

(b) How does the Realm guest kernel's `ioremap` flow differ from a Normal VM's?

A Normal VM's `ioremap()` simply maps the device at its physical address with device attributes — nothing more. A Realm guest's `ioremap()` does one extra step: for ordinary (shared) MMIO it sets the shared (non-secure) bit in the page-table mapping, so accesses are directed to the unprotected half of the guest's IPA space instead of the protected half. Because the device then sits in the unprotected IPA, the RMM allows the access to be forwarded to the host for emulation rather than injecting a fault. In short, the realm flow adds the shared bit to the mapping while the normal flow does not, and that is what keeps the MMIO access from triggering an SEA.

(c) If the realm guest kernel already takes care of `ioremap`, why does `earlycon` still fail?

`earlycon` does not go through the realm-aware `ioremap()` path at all. It sets up its own very early mapping for the UART, and that early path maps whatever physical address the kernel command line gives it verbatim, without the shared-bit step that the realm `ioremap()` would normally add. So even though the realm kernel handles the shared bit for ordinary drivers, `earlycon`'s mapping still points at the protected half of the IPA space. When `earlycon` writes to the PL011, the access lands on a protected IPA and the RMM responds by injecting a Synchronous External Abort into the realm (the repeated "Inject Sync EA"), which is why `earlycon` fails. This is also why the fix must be made in Cloud Hypervisor: we pre-set the shared bit in the `earlycon=` address on the command line, so that even `earlycon`'s verbatim early mapping lands in the unprotected half.

Q6. Host kernel: MMIO fault forwarding

(a) Cite the host KVM function where the fault IPA is extracted and dispatched for MMIO emulation.

The function is `kvm_handle_guest_abort` in `arch/arm64/kvm/mmu.c`. It first extracts the fault IPA with `kvm_vcpu_get_fault_ipa()`, and after determining that the IPA falls in a device region with no memslot, it complements the bottom 12 bits and hands it to `io_mem_abort()` for MMIO emulation:

```
/*
 * The IPA is reported as [MAX:12], so we need to
 * complement it with the bottom 12 bits from the
 * faulting VA. This is always 12 bits, irrespective
 * of the page size.
 */
ipa |= kvm_vcpu_get_hfar(vcpu) & GENMASK(11, 0);
ret = io_mem_abort(vcpu, kvm_gpa_from_fault(vcpu->kvm, ipa));
goto out_unlock;
```

(b) What register does the host use to obtain the faulting IPA of stage-2 data abort? from `arch/arm64/include/asm/kvm_emulate.h`, `kvm_vcpu_get_fault_ipa()`:

```
static __always_inline phys_addr_t kvm_vcpu_get_fault_ipa(const struct kvm_vcpu *vcpu)
{
    return ((phys_addr_t)vcpu->arch.fault.hpfar_el2 & HPFAR_MASK) << 8;
}
```

The register is `HPFAR_EL2` (Hypervisor IPA Fault Address Register). As shown, it takes the `FIPA` field of `hpfar_el2` and shifts it `<< 8` to reconstruct the IPA.

(c) Since `PL011` is emulated in userspace by Cloud Hypervisor, KVM forwards this access to userspace, what is the KVM function that performs this task?

The function is `io_mem_abort` in `arch/arm64/kvm/mmio.c`. It first decodes the syndrome and tries to let an in-kernel device (`kvm_io_bus_read/write`) handle it; if nobody claims it (returns non-zero), it fills `run->mmio` and sets the `exit_reason`, returning to userspace so the VMM can emulate it:

```
/*
 * Prepare MMIO operation. First decode the syndrome data we get
 * from the CPU. Then try if some in-kernel emulation feels
 * responsible, otherwise let user space do its magic.
 */
is_write = kvm_vcpu_dabt_iswrite(vcpu);
len = kvm_vcpu_dabt_get_as(vcpu);
rt = kvm_vcpu_dabt_get_rd(vcpu);

if (is_write) {
    data = vcpu_data_guest_to_host(vcpu, vcpu_get_reg(vcpu, rt),
                                   len);

    trace_kvm_mmio(KVM_TRACE_MMIO_WRITE, len, fault_ipa, &data);
    kvm_mmio_write_buf(data_buf, len, data);
}
```

```

    ret = kvm_io_bus_write(vcpu, KVM_MMIO_BUS, fault_ipa, len,
                           data_buf);
} else {
    trace_kvm_mmio(KVM_TRACE_MMIO_READ_UNSATISFIED, len,
                  fault_ipa, NULL);

    ret = kvm_io_bus_read(vcpu, KVM_MMIO_BUS, fault_ipa, len,
                          data_buf);
}

```

(d) Following (c), what is the exit_reason of the ioctl? The exit_reason is KVM_EXIT_MMIO From arch/arm64/kvm/mmio.c

```

if (!ret) {
    /* We handled the access successfully in the kernel. */
    if (!is_write)
        memcpy(run->mmio.data, data_buf, len);
    vcpu->stat.mmio_exit_kernel++;
    kvm_handle_mmio_return(vcpu);
    return 1;
}

if (is_write)
    memcpy(run->mmio.data, data_buf, len);
vcpu->stat.mmio_exit_user++;
run->exit_reason = KVM_EXIT_MMIO;
return 0;

```

(e) Does the faulting IPA that the userspace VMM receives equal to the value held by the register in (b)? If it is different, why?

From arch/arm64/kvm/mmu.c

- IPA

```
ipa = fault_ipa = kvm_vcpu_get_fault_ipa(vcpu);
```

- VMM receives value

```
ipa |= kvm_vcpu_get_hfar(vcpu) & GENMASK(11, 0);
```

The HPFAR register only reports the faulting address aligned to page size, so the lowest 12 bits (bits[11:0]) are 0. KVM therefore ORs back the bottom 12 bits taken from the faulting VA (`kvm_vcpu_get_hfar(vcpu) & GENMASK(11, 0)`) before handing the IPA to `io_mem_abort` / userspace. The comment in `mmu.c` states this explicitly: “The IPA is reported as [MAX:12], so we need to complement it with the bottom 12 bits from the faulting VA. This is always 12 bits, irrespective of the page size.” So the faulting IPA the VMM receives is different from the raw value of the register in (b) — the difference is the restored bottom-12-bit offset (here, the UARTFR offset that earlycon hits is exactly `0x018`).